

Technical Design Document

PR Description RAG — LangChain JS Capstone

Field	Value
Author	Waldo Panozo
Course	Generative AI for Javascript Developers — LangChain, RAG
Assignment	Strategic Blueprint: From Prototype to Production-Ready LLM Features
Date	June 30, 2026
Repository	portfolio/langchain-js-capstone

1. Executive summary

This document describes **PR Description RAG**, a prototype that drafts pull request descriptions by combining a Jira-style ticket with a git diff. The system uses **Retrieval-Augmented Generation (RAG)** with **LangChain Expression Language (LCEL)** and runs **entirely on local hardware** via Docker and Ollama—no paid API keys required for the default path.

The business problem is real: engineers spend 10–20 minutes per PR writing reviewer context. Automating a first draft that maps acceptance criteria to code changes reduces review cycles and missed requirements.

2. Problem definition and ROI

2.1 Pain point

During sprint delivery, developers often submit PRs with minimal descriptions. Reviewers must read the full diff to understand:

- What ticket is being closed
- Which acceptance criteria are covered

- What to test before approval

This creates friction, especially when diffs span multiple files (middleware, program entry, tests).

2.2 Target user

Backend developer submitting a PR after implementing a story (e.g. rate limiting on a booking API).

2.3 Sample scenario (included in repo)

Ticket DEV-142: Add per-IP rate limits on `POST /api/bookings` with HTTP 429 and `Retry-After`.

Diff: New `RateLimitMiddleware.cs`, registration in `Program.cs`, skeleton unit test.

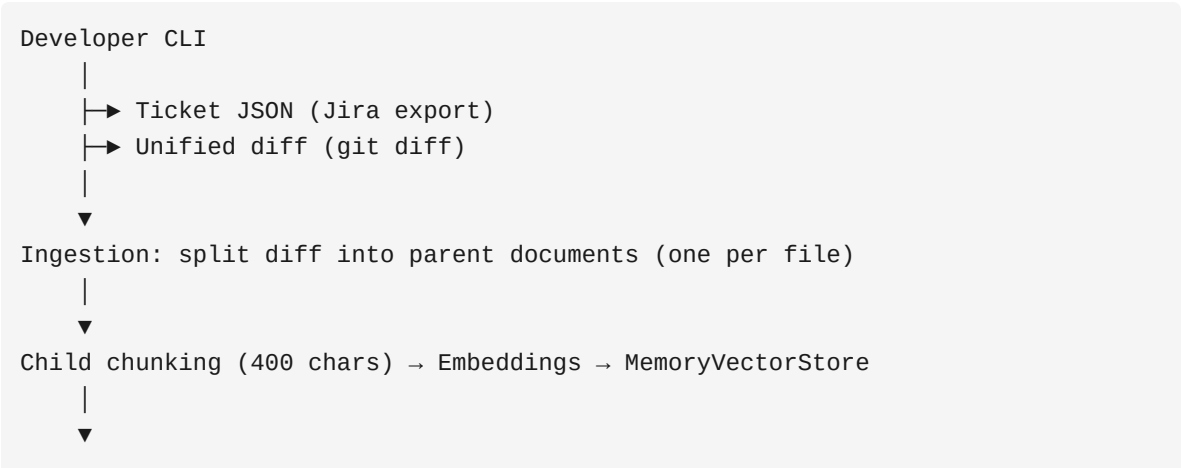
2.4 Return on investment

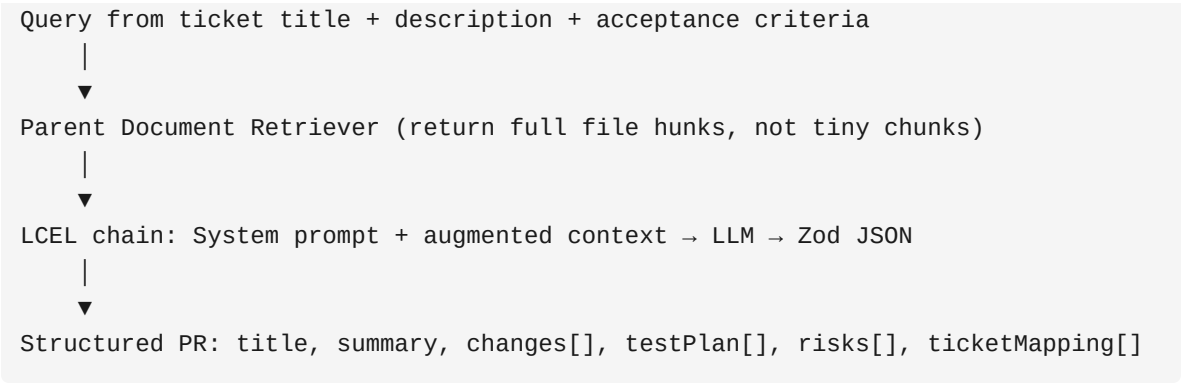
Mode	Cost per generation	Notes
Local (Ollama)	\$0	Default; suitable for demos and privacy-sensitive code
Cloud (OpenAI)	~\$0.03–0.06	Optional; <code>gpt-4o-mini</code> + embeddings

If a draft saves 15 minutes of engineer time, break-even occurs after a handful of PRs per month—even in cloud mode. Local mode removes marginal cost entirely.

3. Solution architecture

3.1 High-level flow





3.2 Required data-flow (assignment)

Per Moodle rubric, the pipeline implements:

User → Embedding → Vector Store → LLM → Output Parser

- **User:** CLI passes ticket + diff paths
- **Embedding:** `nomic-embed-text` (Ollama) or `text-embedding-3-small` (OpenAI)
- **Vector Store:** `MemoryVectorStore` with child chunks
- **LLM:** `qwen2.5:0.5b` (local) or `gpt-4o-mini` (cloud)
- **Output Parser:** Zod schema + coercion layer for tiny local models

3.3 Orchestration layer

We use **LCEL** (`prompt.pipe(llm), RunnableSequence`) as taught in the Udemy course—not a single raw API call. Components are modular:

Module	Responsibility
<code>src/ingest/loadDocuments.ts</code>	Load ticket/diff, build parent documents
<code>src/retrieval/parentDocumentRetriever.ts</code>	Parent Document Retrieval
<code>src/chains/prDescriptionChain.ts</code>	RAG + generation
<code>src/chains/structuredOutput.ts</code>	JSON fallback + Zod validation
<code>src/config.ts</code>	Provider switch: <code>local</code> <code>openai</code>

3.4 Advanced optimization: Parent Document Retrieval

Why: Embedding entire diffs is expensive and noisy. Embedding tiny chunks alone loses file-level context.

How:

1. Each file hunk in the diff is a **parent document**
2. Parents are split into **child chunks** (~400 characters, 80 overlap)
3. Only children are embedded and stored

4. At query time, matching children map back to their **full parent hunk**

This is the "Pro Touch" required by the assignment brief.

4. Key technical decisions

4.1 Why local-first with Docker?

Factor	Decision
Cost	No API billing for capstone demo
Privacy	Diffs may contain proprietary code
Reproducibility	<code>docker compose</code> + pinned models
Assignment	Demonstrates production thinking without cloud dependency

4.2 Model selection (local)

Role	Model	Rationale
Chat	<code>qwen2.5:0.5b</code>	~400 MB; runs on CPU; sufficient for summarization prototype
Embeddings	<code>nomic-embed-text</code>	~270 MB; standard for semantic search with Ollama

4.3 Why not agentic tool-calling?

The use case is **document-grounded summarization**, not multi-step external actions. A RAG pipeline is simpler, cheaper, and easier to defend in a TDD. Agents could be a phase-2 extension (e.g. fetch live Jira via API).

4.4 Structured output strategy

OpenAI supports native `withStructuredOutput(Zod)`. Tiny local models often return malformed JSON (objects inside arrays). We added `coercePrShape()` to normalize responses before Zod validation—documented as iteration evidence.

5. Security and production considerations

5.1 Prompt injection

Diff content is **untrusted**. An attacker could embed instructions in a comment. Mitigation:

- System prompt explicitly ignores instructions inside diff text
- Model only summarizes retrieved context

5.2 Zero retrieval results

If the vector store returns no chunks, the prompt instructs the model to:

- Stay conservative
- Flag missing code context in `risks[]`

5.3 Monitoring

- **Local:** `evidence/run-log.txt` + CLI stdout
- **Optional:** LangSmith (`LANGCHAIN_TRACING_V2=true`) for trace screenshots

5.4 Production roadmap

Concern	Prototype	Production
Vector store	In-memory	Pinecone / pgvector
Auth	N/A	API keys, SSO
Model routing	Single tiny model	Escalate large diffs to larger model
CI integration	CLI	GitHub Action on <code>git diff</code> + Jira webhook

6. Iteration and test evidence

6.1 Test environment

Verified on **June 30, 2026** with:

```
npm run docker:demo
```

- Docker: `Ollama latest` + Node 20 app container
- Models: `qwen2.5:0.5b`, `nomic-embed-text`
- Index: 4 parent documents, 12 child chunks

6.2 Iteration v1 — unstructured prompt

Configuration: High temperature (0.7), free-form markdown output.

Observed issues:

- Long prose with invented code snippets not present in the diff
- Hallucinated API details (e.g. wrong middleware structure)
- Hard to paste into GitHub PR template

Artifact: `evidence/iteration-v1-raw.txt`

6.3 Iteration v2 — structured pipeline

Configuration: Temperature 0.1, Zod schema, injection guard, explicit AC mapping, JSON coercion for local models.

Improvements:

- Valid JSON with `title`, `summary`, `changes`, `testPlan`, `risks`
- Correct mention of HTTP 429 and Retry-After from ticket/diff context
- No invented C# code blocks

Limitations (expected with 0.5B model):

- Incomplete changes list (2 of 4 AC items)
- `ticketMapping` nested JSON string instead of clean AC mapping

Artifact: `evidence/iteration-v2-structured.json`

6.4 Parser hardening (engineering fix)

First v2 run failed with `ZodError` because the tiny model returned objects inside arrays. Fix: `coercePrShape()` in `structuredOutput.ts`—demonstrates test → fail → improve cycle required by rubric.

7. Trade-off analysis

Trade-off	Choice	Alternative	Justification
Quality vs cost	Local 0.5B model	gpt-4o	Zero cost; good enough for prototype
Latency vs context	Parent retrieval	Full diff in prompt	Fewer tokens, better focus
Strict JSON vs flexibility	Zod + coercion	Free text only	Enables PR template automation
Memory vs persistence	MemoryVectorStore	Persistent DB	Simpler for capstone; swap later

8. Rubric alignment

Criterion (weight)	Evidence in this project
Problem & logic (35%)	DEV-142 rate-limiting scenario, ROI table
Architecture (20%)	Modular LCEL, RAG, Parent Document Retrieval
Implementation (20%)	Docker CLI, sample data, edge-case prompts
Iteration (15%)	v1 vs v2 artifacts, Zod coercion fix
Prod-readiness (10%)	Security, monitoring plan, Docker deployment

9. How to reproduce

```
cd portfolio/langchain-js-capstone
npm run docker:demo
```

Outputs:

- evidence/iteration-v1-raw.txt
- evidence/iteration-v2-structured.json
- evidence/run-log.txt

See README.md for full setup options (host Node + Docker Ollama, optional OpenAI).

10. Conclusion

PR Description RAG demonstrates architectural thinking beyond a single LLM API call: ingestion, embeddings, retrieval optimization, LCEL orchestration, structured output, and a Docker-based local deployment path. The iteration from v1 to v2 shows measurable quality improvement and honest limits of tiny on-device models—exactly the kind of engineering judgment the capstone assignment expects.